DressMe

Technical presentation deck

8 LangGraph agents, including one conversational orchestrator

4 FastAPI services, 1 cited RAG, A2A 0.3.0 protocol

Architecture, LLMs, inputs / outputs, graphs, algorithms, limits

---

Architecture, agents, LLMs, graphs and algorithms

Final-year project - React 19, FastAPI, LangGraph, OpenAI, Supabase

1. Executive summary

DressMe is a smart wardrobe application. The user photographs garments, the app catalogues them automatically (YOLO detector + GPT-4o vision), and eight LangGraph agents then help the user get dressed, build outfits, pack a suitcase, find wardrobe gaps and decide on a purchase. A single orchestrator provides one conversational entry point.

The architectural principle is constant: **the LLM decides and writes, deterministic code verifies and computes**. No agent can cite a garment that is not in the wardrobe, invent a weather forecast, or cite a source that does not exist in the RAG.

Indicator	Value
LangGraph agents	8 (including 1 orchestrator) plus 2 alternative orchestrators
FastAPI services	BO1 8000, BO3 8001, Help Me Pack 8020, RAG 8010, orchestrator 8070
LLM models	GPT-4o, gpt-4o-mini, Qwen2.5-3B + LoRA, Ollama qwen2.5:3b
RAG corpus	38 PDFs, 14,574 chunks, all-MiniLM-L6-v2 (384 dimensions)
RAG quality	Hit@5 91.67%, MRR 85.42%, context precision 71.67%
Tests	101 orchestrator, 85 backend, 30 frontend, 1,095 BO4 (25 skipped)
Agent-to-agent protocol	A2A 0.3.0 over JSON-RPC 2.0
Garment categories	20 detected and catalogued classes
Trip Agent tools	10 Pydantic-validated tools + OR-Tools CP-SAT solver

What makes the project distinctive

- **Strong grounding**: garment ids always come from the real wardrobe; weather comes from Open-Meteo; fashion advice comes from books, with [S#] markers.
- **Code verification**: an outfit is validated by rules (dress alone or top + bottom, shoes required, no duplicate role), and a purchase verdict is validated by code.
- **Graceful degradation**: if the RAG, weather, LLM or Hugging Face is unavailable, a deterministic fallback keeps the page usable.
- **Standard communication**: every agent publishes an A2A card and accepts JSON-RPC tasks, so the orchestrator can delegate without knowing internal routes.

2. The product: eleven screens, eight agents

Each interface page calls a route; the route calls either an **agent** (the LLM chooses tools or follows a graph) or an **engine** (detector, purchase engine, local comparison).

Screen	Route	Agent / engine	Input	Output
AI Stylist + voice	POST /api/agent/search	AI Stylist (LangGraph)	message, city, country, unit	advice, weather, outfit, reasons, sources
Wardrobe	POST /api/wardrobe/agent	Wardrobe Agent (LangGraph)	message, action	items, filters, proposal
Outfit Builder	POST /api/outfits/agent	Outfit Builder (LangGraph)	occasion, season, city, avoid	3 valid outfits + sources
Help Me Pack	POST :8020/api/trips	Trip Agent + CP-SAT	message, session_id	suitcase, checklist, alternatives
Wardrobe Gaps	POST :8001/api/bo3/agent	Gap Agent + XGBoost	message, goal	ranked gaps, covered items
Live scan	/api/items/quick-detect + check-duplicate	YOLO + Wardrobe Agent	camera stream	boxes, quality, duplicate
Should I buy this?	POST /api/shopping-assistant /evaluate	Qwen + 6 tools + GPT-4o	item photo	BUY / CONSIDER / SKIP verdict
Find Similar	POST /api/items/analyze (all_items)	YOLO + local comparison	outfit photo	owned / close / missing
Add, wardrobe, 3D	/api/items/analyze, /api/items	YOLO + vision cataloguing	photo	garment record, 3D model
Dashboard	/api/items + :8001 gaps	Gap Agent + local rules	none	statistics, outfit of the day
Avatar, try-on	/api/avatar/*, tryon-outfit	WaveSpeed / fal.ai	photo, outfit	generated image, job status

Agents no longer filter the wardrobe themselves: they **ask the Wardrobe Agent** in natural language, either by a direct call inside BO1 or by A2A for the Trip Agent. Three LLM components are not agents: vision cataloguing, purchase-evidence computation and RAG answer generation.

3. Global architecture

Four independent FastAPI services are called by the React 19 interface. Supabase provides authentication, the database and storage. OpenAI provides the LLMs and Open-Meteo the real weather.

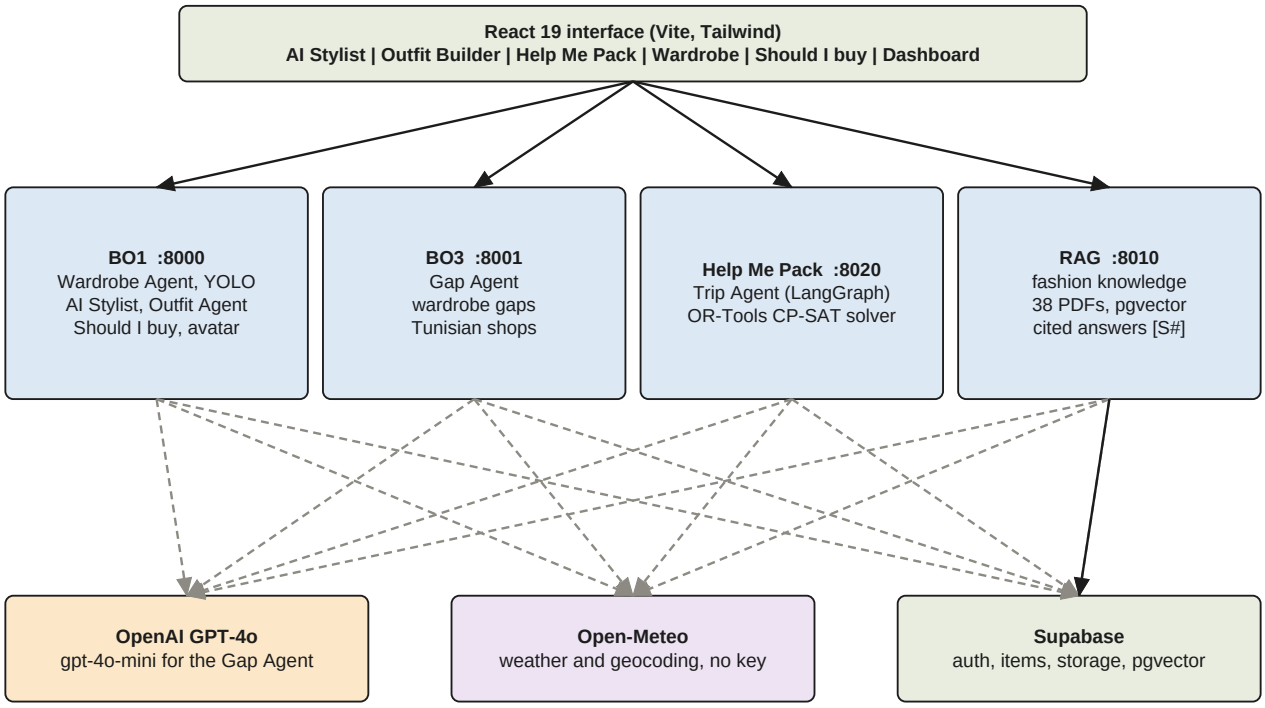


Figure 1: services, ports and external dependencies.

Service	Port	Stack	Role
BO1 (backend/app)	8000	FastAPI, LangGraph, YOLO	wardrobe, detector, AI Stylist, Outfit Agent, Similar Agent, Purchase Agent, avatar
BO3 (DressMe UI Blueprint)	8001	FastAPI, gpt-4o-mini, XGBoost	Gap Agent, Tunisian shop products
Help Me Pack	8020	FastAPI, LangGraph, OR-Tools	Trip Agent, CP-SAT solver, checklist
RAG (rag_backend)	8010	FastAPI, pgvector, MiniLM	cited fashion knowledge, [S#] answers
Standalone orchestrator	8070	FastAPI, LangGraph, Ollama	legacy orchestrator, kept as a local alternative
Interface (Vite)	8443	React 19, Tailwind	web application, development https
Ollama	11434	qwen2.5:3b	local LLM for BO4 Stylist and the 8070 router

4. Data, authentication and memory

Every request carries a Supabase token. A FastAPI dependency verifies it and exposes a **user_id**; all SQL queries are filtered by that id. A user can therefore never read or modify another user's wardrobe, including through an A2A task.

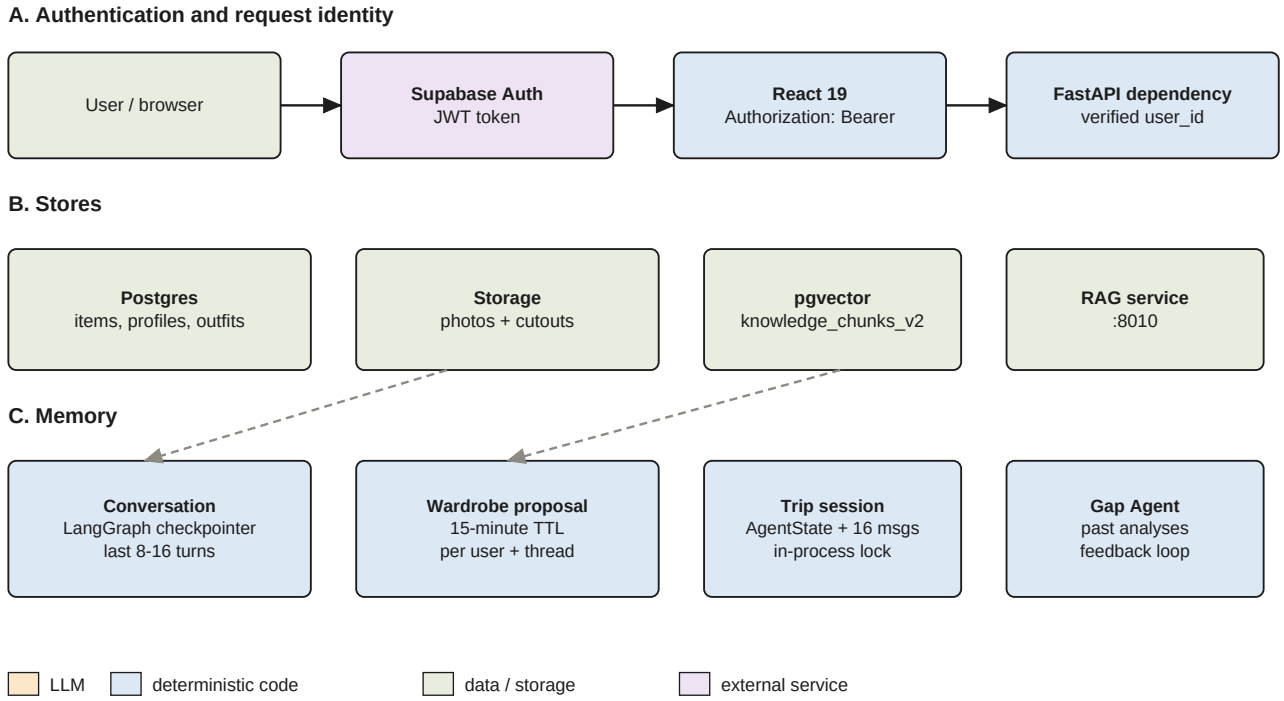


Figure 2: request identity, storage and memories.

Storage

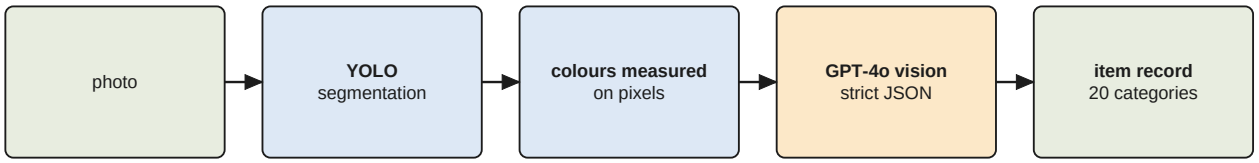
Store	Content	Used by
Postgres items	garment record: category, colour, material, season, occasion, style, warmth, image	all agents
Postgres profiles	city, country, unit, preferences, department	AI Stylist, Outfit, Gap
Storage	original photos and transparent cutouts	interface, try-on, 3D
pgvector knowledge_chunks_v2	14,574 passages + 384-d embeddings, HNSW cosine index	RAG service
Outfits, feedback	saved outfits, feedback on suggestions	Outfit Builder, Gap Agent

Agent memories

Memory	Scope	Duration
LangGraph checkpointer	one thread per user and conversation; last 8 turns	persisted server-side
Wardrobe Agent	pending correction proposal (yes / no)	15 minutes, bound to the user
Trip Agent	full AgentState + last 16 messages	in-process session, lock per session_id
Gap Agent	past analyses and user feedback	database
RAG	conversation history	service session

5. Vision pipeline: from photo to record

Cataloguing is a two-stage pipeline. The local detector does the geometric and colour work; the vision LLM does the semantic work.



Detector first, LLM second: the model refines but never invents a garment that is not in the photo.

No API key or model error: deterministic fallback (fallback_enrich) still creates a usable item.

Figure 3: garment cataloguing pipeline.

- **YOLO** (`dressme_best.pt`) detects and segments the garment in the photo.
- **Colours are measured on the mask pixels** and mapped to the nearest palette entry: the LLM does not guess the colour.
- **GPT-4o vision** receives the image and a strict JSON schema: category, material, season, occasion, style, formality, warmth level, confidence. It may only override the detector category when the image clearly shows it.
- Without an API key or on a model error, `fallback_enrich` applies rules and the record remains usable.
- 20 classes: t-shirt, shirt, sweater, jacket, coat, blouse, dress, skirt, trousers, jeans, shorts, jumpsuit, shoes, boots, sandals, bag, belt, hat, scarf, accessory.

6. LLMs and models used

The model choice depends on the task: GPT-4o for reasoning and vision, gpt-4o-mini for a short and cheap task, Qwen + LoRA for a specialised decision, Ollama to run without an API key.

Component	Model	Temp.	Format	Limit	Role
Vision cataloguing	GPT-4o	-	strict JSON	1 call	category, colour, material, season, occasion
AI Stylist	GPT-4o	0.3	JSON	8 rounds, 1,800 tokens	advice, weather, RAG, wardrobe
Wardrobe Agent	GPT-4o	0.1	JSON	6 rounds	search, count, duplicates, correction
Similar Agent	GPT-4o	0.1	JSON	10 rounds	owned / close / missing statuses
Outfit Builder (compose)	GPT-4o	0.6	JSON	2 passes, 2,200 tokens	3 outfits with real ids
Trip Agent	GPT-4o	0	tools + JSON	12 iterations	tool choice, CP-SAT suitcase
Gap Agent	gpt-4o-mini	0.2	Markdown	6 steps	wardrobe gaps
Purchase Agent	Qwen2.5-3B + LoRA	0.1	JSON	6 rounds	BUY / CONSIDER / SKIP verdict
Purchase (writing)	GPT-4o	0.5	JSON 3 sections	-	rewrites the verdict, does not decide
Purchase Redundancy	GPT-4.1 / Claude Opus 5.5	0.2	JSON	1 call	redundancy + CLIP ViT-B/32
BO4 Stylist	Ollama qwen2.5:3b	0.2	JSON	9 nodes	local outfits, heuristics
Orchestrator 8070	Ollama qwen2.5:3b	0	JSON / rules	1	intent routing
RAG search	MiniLM + LLM	0.1	cited text	top 5	sourced answer [S#]

7. Inputs and outputs per feature

All routes are protected by the Supabase token. Responses are JSON; the interface then renders dedicated areas (weather, outfits, reasons, sources, verdict).

Feature	Input	Main output
AI Stylist	{message, city, country, unit}	{answer, weather, outfit[], why[], alternatives[], tips[], sources[], items[]}
Wardrobe	{message, action?, confirm?}	{reply, items[], filters, proposal{fields, item_id}}
Similar	{pieces[]} via A2A	{pieces[{status: owned|close|missing, match_item, missing_spec}]}
Outfit Builder	{occasion, season, city, avoid[]}	{outfits[{title, items[], weather_note, why[]}], missing[], sources[]}
Trip Agent	{message, session_id}	{message, state, capsule[], checklist[], alternatives[]}
Gap Agent	{message, goal}	{answer, gaps[], covered[], products[]}
Purchase Agent	{image} or photo	{decision, score, reasons[], matching_item_ids[], decision_source}
Avatar	{photo} / {outfit_id}	{job_id, status, result_url}

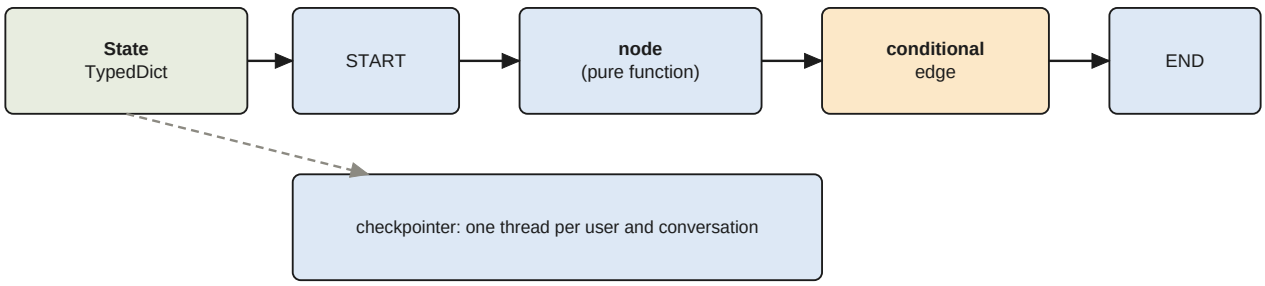
Example AI Stylist response

```
{
  "answer": "Wear the beige blazer with the white shirt and navy trousers.",
  "weather": "27 C and sunny in Tunis; light layers are enough.",
  "outfit": ["Beige blazer + White shirt + Navy trousers + Leather loafers"],
  "why": ["The blazer keeps the dinner look formal [S2].", "..."],
  "alternatives": ["The grey cardigan is too casual for a dinner."],
  "tips": ["Choose breathable cotton for a warm evening [S4]."]
}
```

8. LangGraph: how an agent works

LangGraph models an agent as a **state machine**. A shared state (a TypedDict) flows through nodes; each node is a Python function that reads the state and returns a partial update. Conditional edges choose the next node. A checkpointer persists the state between calls.

A. Anatomy of a LangGraph state machine



B. Shared tool loop (agent_kit/tool_loop.py)

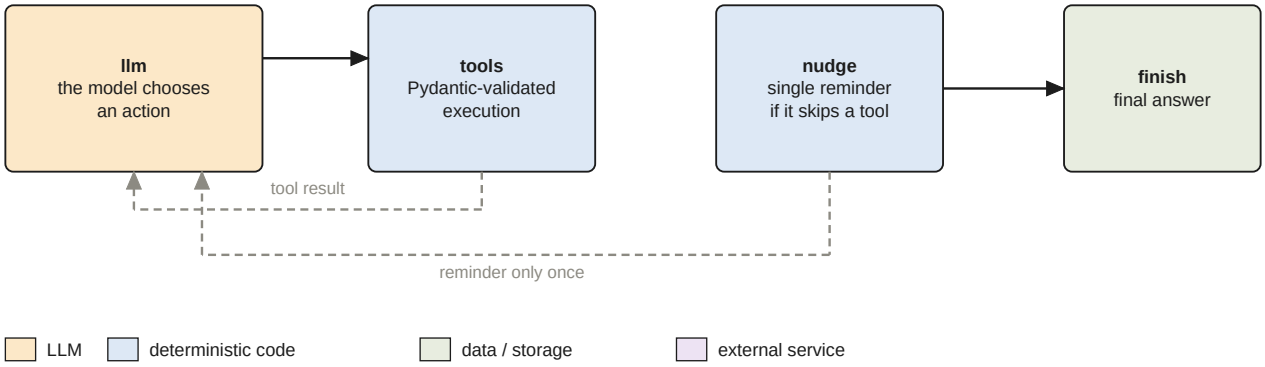


Figure 4: anatomy of a graph and the shared tool loop.

The four building blocks

Block	Role in DressMe
Shared state	messages, items, weather, advice, outfit draft, errors, conversation ids
Node	a pure function: call a tool, build the closet, compose, validate, write
Conditional edge	code chooses the path (e.g. gate -> apply / cancel / assist)
Checkpointer	memory per user and thread; resumes a conversation after InputRequired

The shared tool loop

The module `agent_kit/tool_loop.py` gives every agent the same mechanics: the LLM chooses a tool, the code validates the arguments and runs the tool, and the result returns to the LLM. A **nudge** node sends a single reminder when the model skips a mandatory tool (weather, RAG, wardrobe). A **finish** node stops the loop when the answer is usable.

Every agent reuses this skeleton: an **llm** node, a **tools** node, a conditional edge to **finish** and a return edge **tools -> llm**. Only the model, the tools and the guardrails change from one agent to another.

9. The graphs, one by one

Each agent has its own graph. Nodes are named exactly as in the code, so this document can be read alongside the source files.

9.1 AI Stylist: tool loop

GPT-4o, temperature 0.3, JSON output, 8 rounds maximum. Three tools: `search_wardrobe`, `get_weather`, `fashion_advice`.

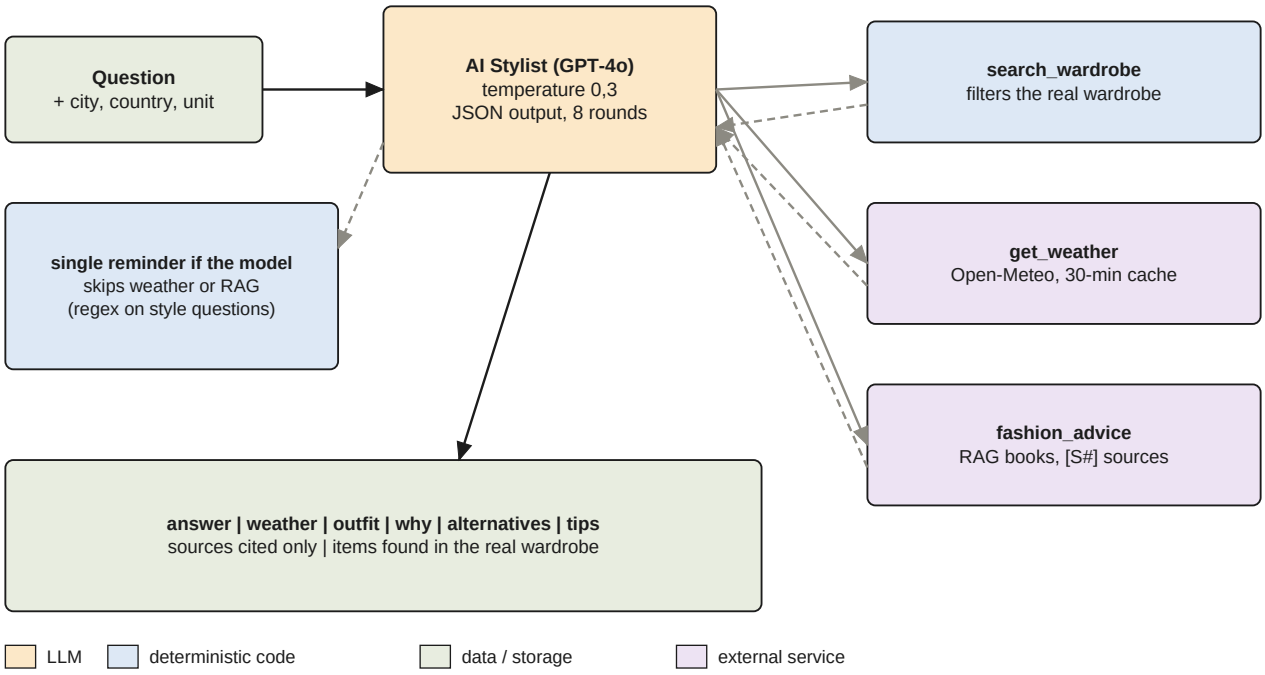


Figure 5: AI Stylist tool loop.

- The prompt requires **search_wardrobe** before answering and forbids mentioning garments the user does not own.
- Any style question triggers **fashion_advice**; any weather question triggers **get_weather**.
- A single reminder is sent if the model forgets weather or RAG (regex detection in FR / EN).
- Only sources actually cited are returned; outfit items are matched by name in the wardrobe.

9.2 Wardrobe Agent: confirmation graph

The Wardrobe Agent owns the wardrobe. Its graph starts with a **gate** node: if a correction is pending, yes leads to **apply** and no to **cancel**; otherwise **assist** starts the tool loop. No LLM tool writes: `update_item` only proposes.

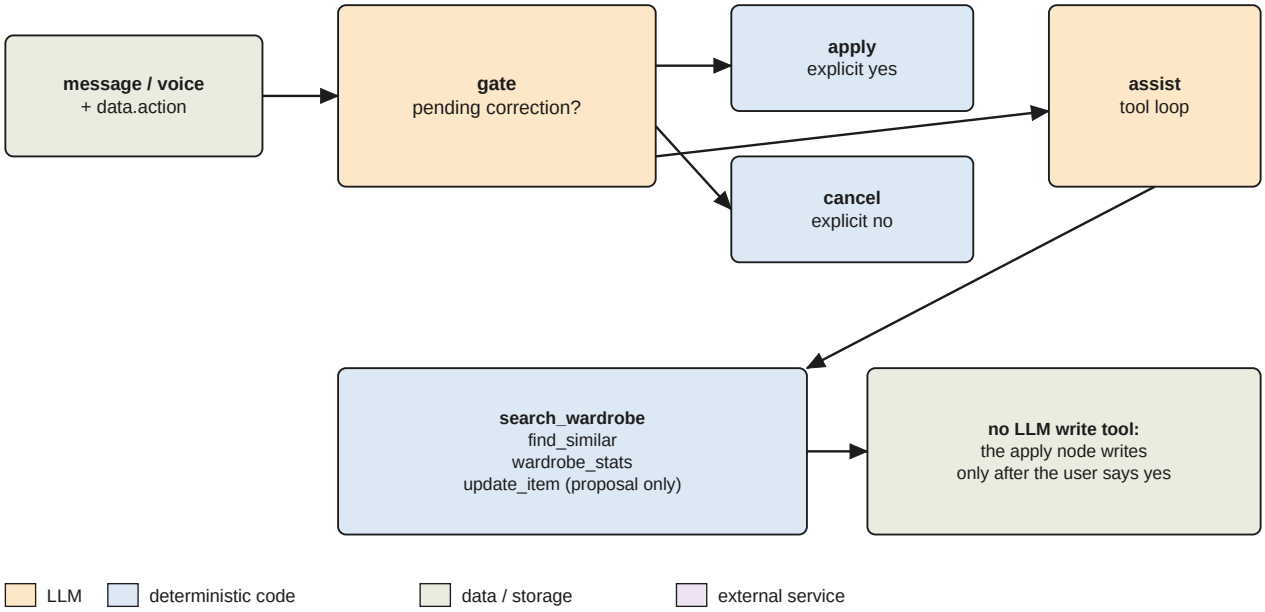


Figure 6: Wardrobe Agent gates and tools.

Tool	Role
search_wardrobe	filters the wardrobe (category, colour, season, occasion, style, warmth, text)
find_similar	close items with reasons (same type, same colour, same material)
wardrobe_stats	total, distribution, items to review
update_item	proposes a correction, no write

A correction is applied only by the **apply** node, after an explicit yes. The proposal expires after 15 minutes and is bound to the user and conversation. The same agent exposes structured actions without an LLM (list, search, check-duplicate) used by other agents and by Live scan.

9.3 Similar Agent: owned, close, missing

From an outfit photo, the detector splits the pieces; the agent asks the Wardrobe Agent (A2A) for close items, proposes a status per piece, and **code verifies** every cited id.

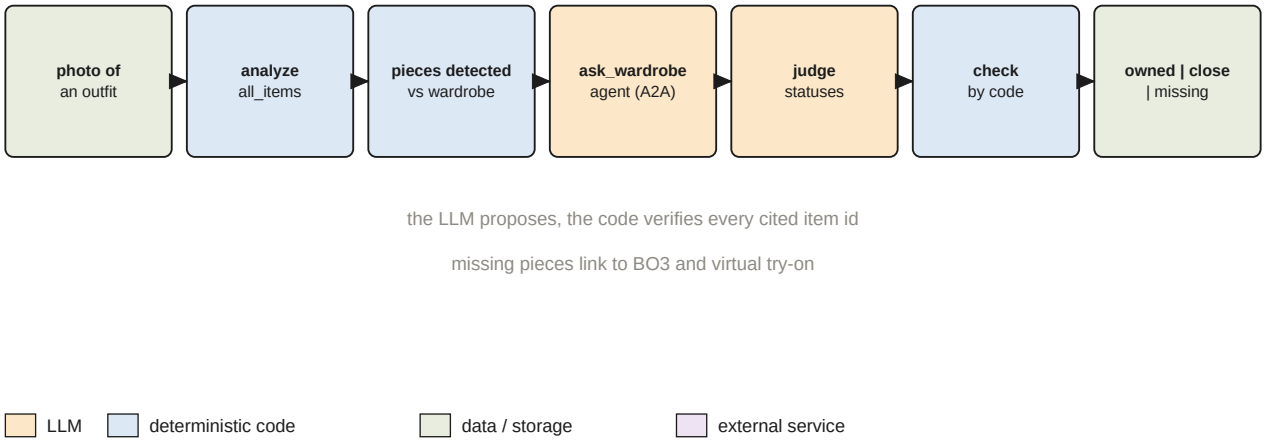


Figure 7: Similar Agent chain.

- **owned**: the exact piece is in the wardrobe; **close**: same category and close colour; **missing**: no match.
- Code validates the statuses and rejects any id absent from the wardrobe.
- A missing piece links to BO3 (Tunisian shop photos) and to virtual try-on.

9.4 Outfit Builder: five-node business graph

The LLM only acts in **compose**. The other four nodes are deterministic code, which guarantees that weather, wardrobe and validation do not depend on the model.

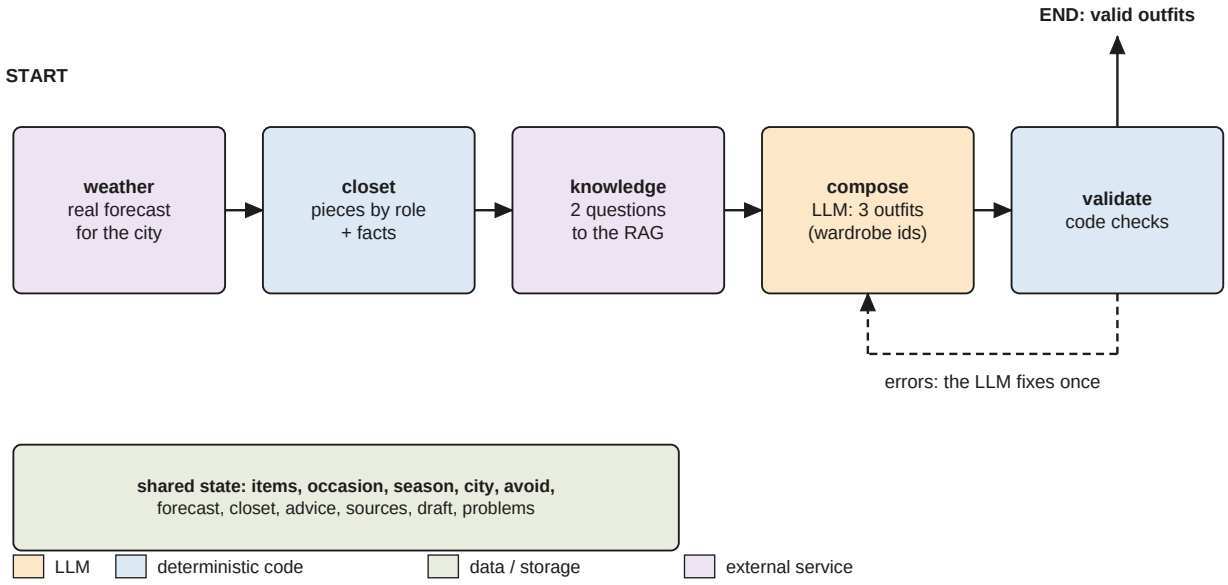


Figure 8: Outfit Builder graph.

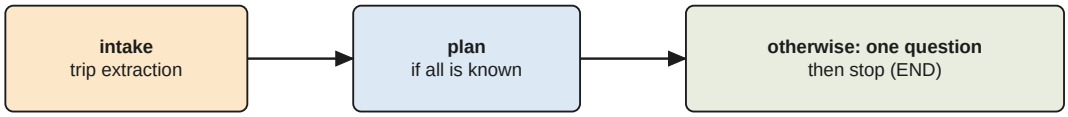
Node	Input	Output	Type
weather	city, country	forecast or None	Open-Meteo
closet	items	pieces grouped by role (13 fields)	code
knowledge	occasion, season, forecast	advice[] + sources	RAG, 2 questions
compose	full brief	outfits[] {title, items[], weather_note, why[]}, missing[]	GPT-4o, JSON, temp. 0.6
validate	outfits[] + wardrobe	valid outfits or errors (back to compose once)	code

Validation rules: real ids; dress alone or top + bottom; shoes required; never two pieces of the same role; a jacket if cold or rainy. On failure the LLM is re-run **once only** with the exact list of errors.

9.5 Trip Agent: two graphs and ten tools

The Trip Agent combines a conversation (graph 1) and a deterministic workflow (graph 2). The LLM invents neither destination, nor dates, nor weather, nor solver result; it chooses the next tool.

Graph 1: conversation (LangGraph)



Graph 2: deterministic workflow (each node stops if the previous step fails)



GPT-4o tool loop (temperature 0, 12 iterations max): 10 tools validated by Pydantic

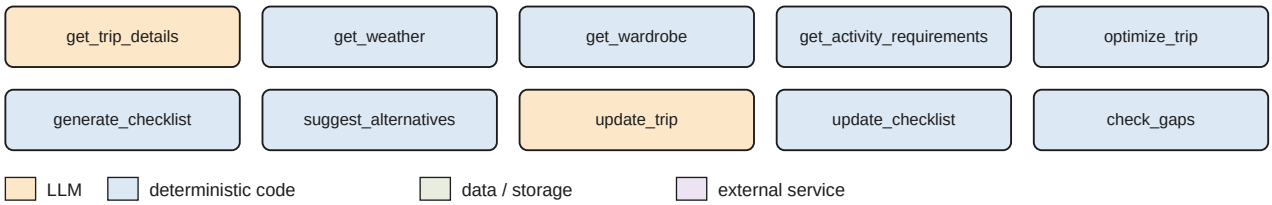


Figure 9: the two Trip Agent graphs and its ten tools.

Tool	Role	Resource
get_trip_details	extracts, merges and validates trip fields	GPT-4o JSON + regex
get_weather	day-by-day forecast for the destination	Open-Meteo
get_wardrobe	loads the wardrobe and role coverage	CSV + pandas
get_activity_requirements	turns activities and weather into requirements	rules
optimize_trip	selects the suitcase (fewer pieces, more combinations)	OR-Tools CP-SAT
generate_checklist	checklist from the capsule	Pydantic
suggest_alternatives	diagnoses INFEASIBLE without relaxing constraints	CP-SAT
update_trip	changes the trip and invalidates stale state	agent state
update_checklist	ticks an item without re-running the solver	agent state
check_gaps	calls the Gap Agent over A2A (optional)	injected function

9.6 Gap Agent: gaps and XGBoost ranking

gpt-4o-mini, temperature 0.2, 6 steps maximum. **Deterministic rules** decide whether a gap exists; XGBoost only ranks it. The prompt states that the ranker is trained on synthetic data until real feedback arrives.

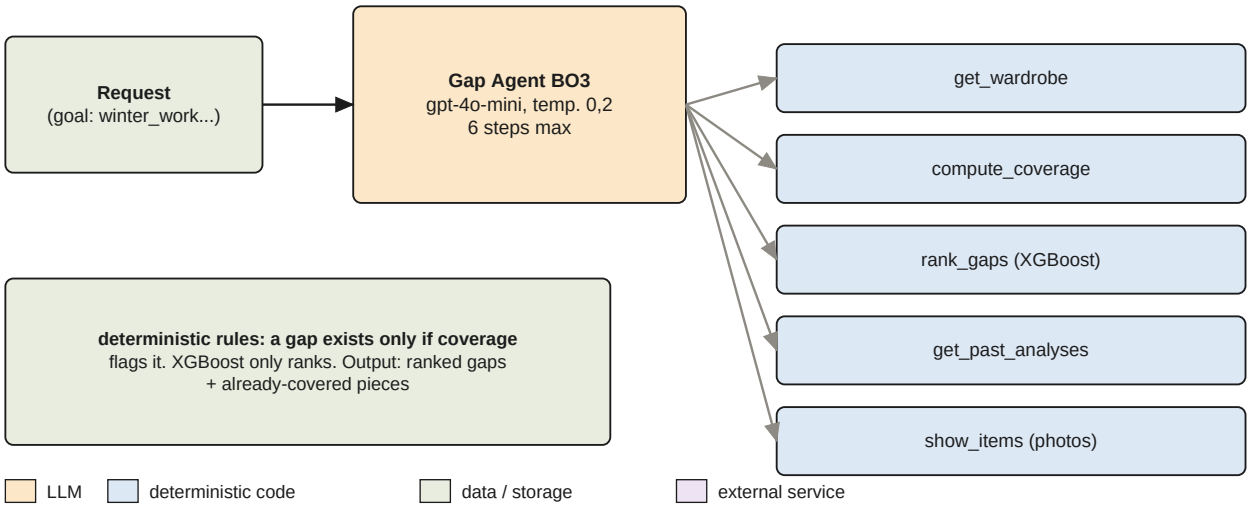


Figure 10: Gap Agent and its five tools.

- **compute_coverage** compares the chosen goal with owned pieces, by season and requirement.
- **rank_gaps** scores each missing item (deficit, usage frequency, season, outfit gain, preference, FashionCLIP redundancy) and ranks with XGBoost.
- **get_past_analyses** provides long-term memory; feedback feeds `/api/bo3/retrain`.
- Never a piece from another department; never a guessed gender. The answer uses the user's language.

9.7 Purchase Agent: Qwen decides, GPT writes

The decision maker is **Qwen2.5-3B-Instruct + LoRA adapter** (`meriemgharbi123/dressme_qwen3b_lora`), loaded from Hugging Face. It chooses six tools in a LangGraph loop; every tool is computed by code on the real wardrobe and its results are authoritative.

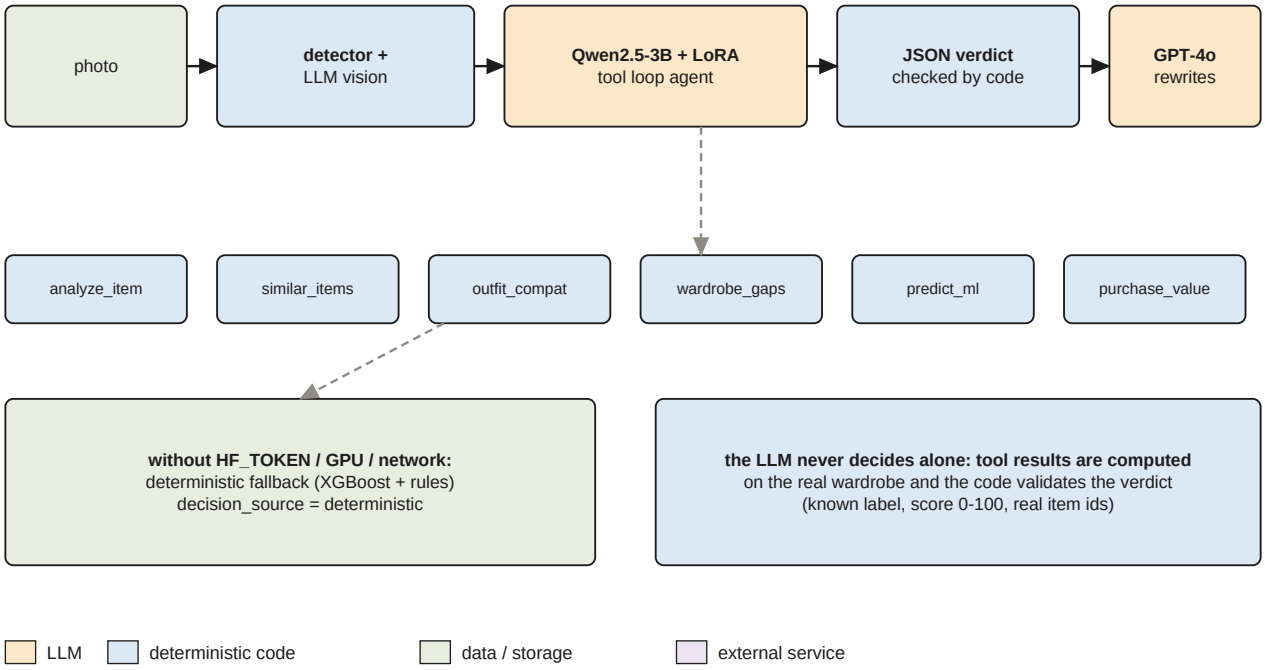


Figure 11: Purchase Agent decision chain.

Tool	What it computes
analyze_item	category, colour and attributes of the photographed item
find_similar_items	close pieces already owned
check_outfit_compatibility	possible outfits with the item
analyze_wardrobe_gaps	whether it fills a gap
predict_purchase_ml	XGBoost purchase-value score
calculate_purchase_value	estimated cost / usage ratio

Qwen ends with a JSON verdict {decision, score, matching_item_ids, reasons}. Code verifies it: known label, score between 0 and 100, ids actually present, short reasons. GPT-4o receives that JSON and rewrites it in three sections without changing the decision. Deterministic fallback when Hugging Face, GPU or network are unavailable; `decision_source` reports the source.

9.8 BO4 Stylist: local nine-node graph

BO4 is a separate stack, not wired to the React interface. It demonstrates a local variant with no API key: Ollama qwen2.5:3b and a heuristic outfit engine.

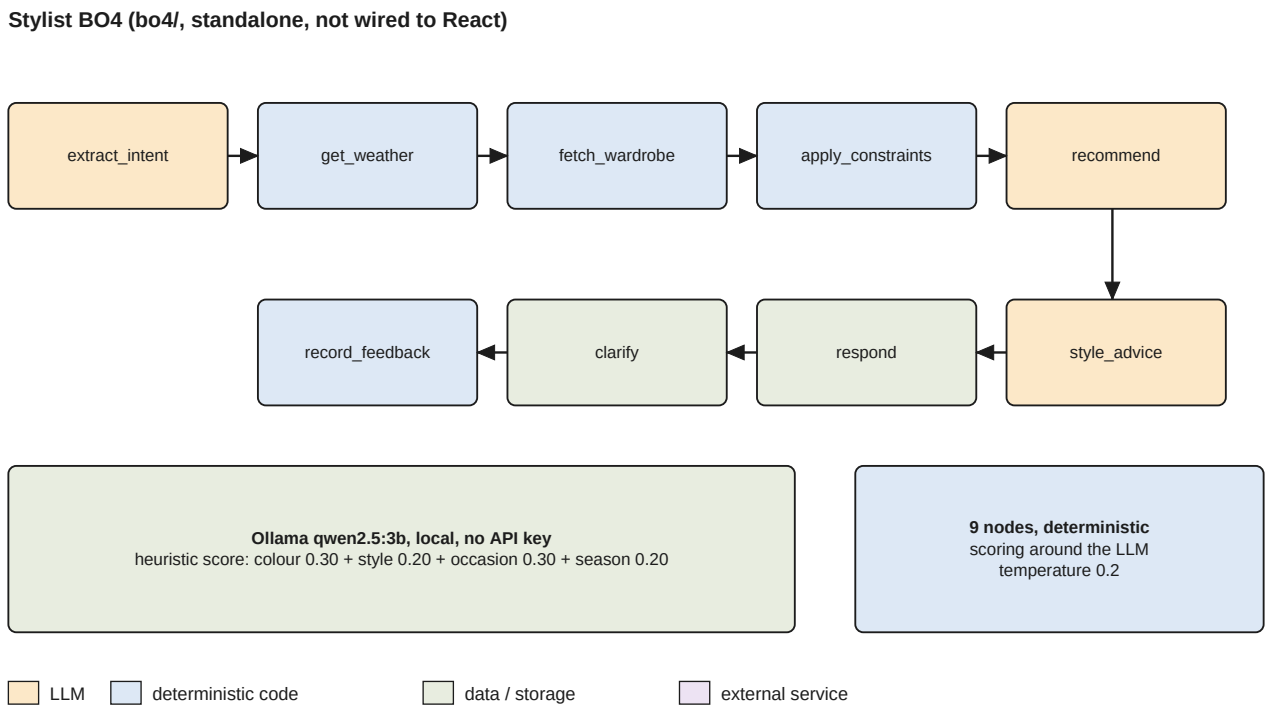


Figure 12: BO4 Stylist graph.

The nine nodes: extract_intent, get_weather, fetch_wardrobe, apply_constraints, recommend, style_advice, respond, clarify, record_feedback. The heuristic score combines colour (0.30), style (0.20), occasion (0.30) and season (0.20).

10. Agent-to-agent communication: A2A

Every agent exposes an **agent card** (identity, skills, authentication) and a **JSON-RPC** endpoint compliant with the Agent2Agent 0.3.0 protocol. Any client can therefore discover an agent and assign it a task over HTTP without knowing its internal routes.

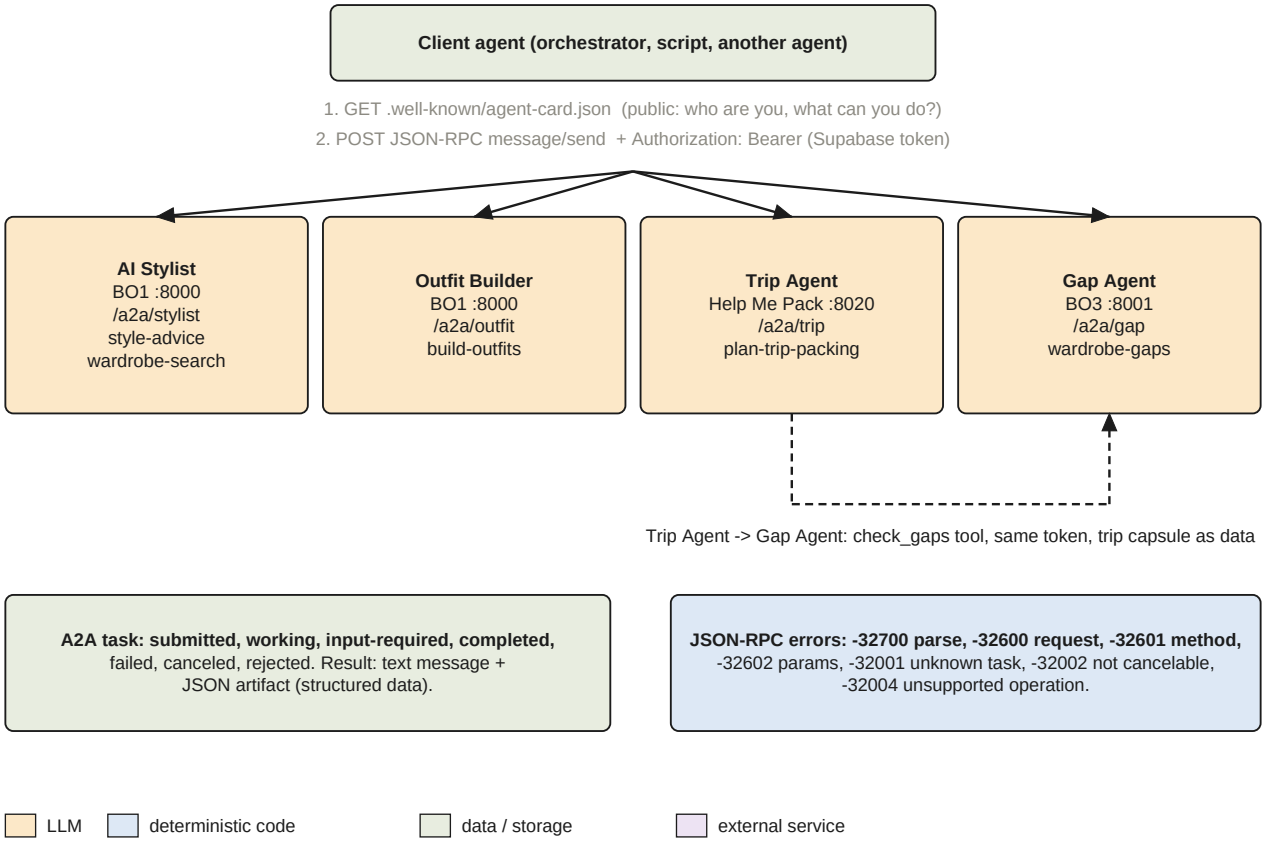


Figure 13: card discovery, task submission, agent to agent.

Agent	Card	Endpoint	Skill
AI Stylist	/a2a/stylist/.well-known/agent-card.json	POST /a2a/stylist (8000)	style-advice, wardrobe-search
Outfit Builder	/a2a/outfit/.well-known/agent-card.json	POST /a2a/outfit (8000)	build-outfits
Wardrobe Agent	/a2a/wardrobe/.well-known/agent-card.json	POST /a2a/wardrobe (8000)	search, list, correction, stats, duplicates
Similar Agent	/a2a/similar/.well-known/agent-card.json	POST /a2a/similar (8000)	find-similar
Trip Agent	/a2a/trip/.well-known/agent-card.json	POST /a2a/trip (8020)	plan-trip-packing
Gap Agent	/a2a/gap/.well-known/agent-card.json	POST /a2a/gap (8001)	wardrobe-gaps

Methods and states

- **message/send** (synchronous response), **tasks/get**, **tasks/cancel**.
- Task states: submitted, working, input-required, completed, failed, canceled, rejected.
- JSON-RPC errors: -32700 parse, -32600 request, -32601 method, -32602 params, -32001 unknown task, -32002 not cancelable, -32004 unsupported operation.
- The card is public; every task requires the Supabase token and only works on that user's wardrobe. A task can only be read by its creator.
- **input-required**: the Trip Agent asks for a missing detail; the client answers with the same `contextId` (trip session id).

- Not implemented and declared as such in the card: streaming (message/stream) and push notifications.

Example task sent to the Wardrobe Agent:

```
POST /a2a/wardrobe
Authorization: Bearer <supabase_jwt>

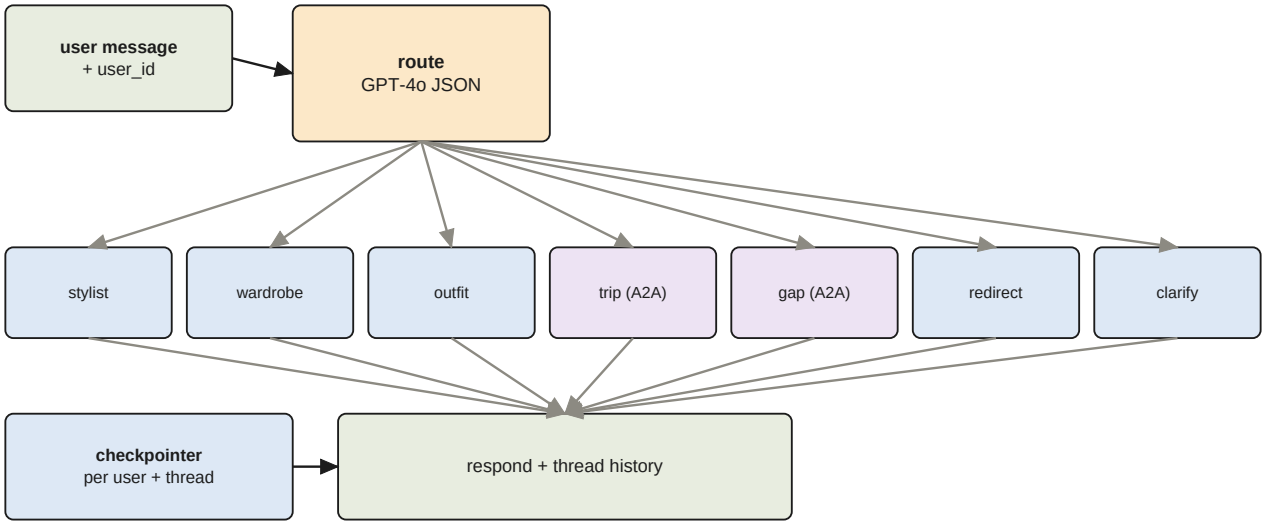
{"jsonrpc": "2.0", "id": "42", "method": "message/send",
 "params": {"message": {"role": "user", "parts": [{"kind": "text", "text": "list my jackets"}]},
            "metadata": {"action": "list"}}}
```

Agent to agent: the Trip Agent's **check_gaps** tool sends the trip capsule to the Gap Agent (A2A, same token). If the Gap Agent is down, the tool returns a clean error and the trip continues. The address is configured with `GAP_A2A_URL`.

11. Orchestrators

The BO1 orchestrator is the conversational entry point of the application: `POST /api/assistant/chat`. It chooses the agent, delegates the request and keeps the thread. Other screens still call their agent directly.

A. BO1 orchestrator: POST /api/assistant/chat



B. Standalone orchestrator (orchestrator/, port 8070)

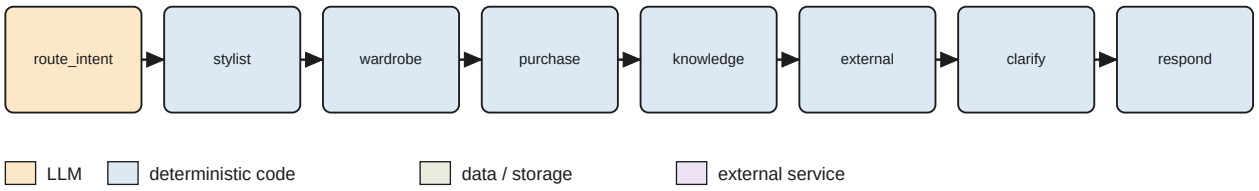


Figure 14: BO1 orchestrator and standalone orchestrator 8070.

BO1 node	Role
route	GPT-4o JSON with a closed list of agents; FR / EN rules if the LLM is unavailable; if an agent is waiting, the next turn goes to the same agent
stylist / wardrobe / outfit	direct call inside BO1
trip / gap	A2A call with the user token (8020 and 8001)
redirect	Should I buy and Find Similar need a photo: opens the right page
clarify	asks for a detail when the request is too vague
respond	appends the turn to history (last 8 turns, one thread per user)

Follow-up conversation example

- "Pack for Paris": routed to the Trip Agent, task **input-required** ("which dates?").
- "June 3 to 7, 8 kg": no agent keyword, but the question is still open; the answer goes back to the Trip Agent with the same contextId.
- A Wardrobe Agent correction is continued only by yes or no; any other sentence is a new request.
- Every answer shows which agent replied ("Answered by ..."); outfits render with photos and a button opens the requested page.

Standalone orchestrator 8070

The `orchestrator/` graph (port 8070) is kept as a local alternative: **route_intent** (Ollama qwen2.5:3b then rules), branches stylist, wardrobe, purchase, knowledge, external, clarify, then respond. It is not wired to the current React interface.

12. The RAG: cited fashion knowledge

The RAG answers style questions from 38 fashion books. It does not know the wardrobe: the agent makes the link between the book rule and the real garments.

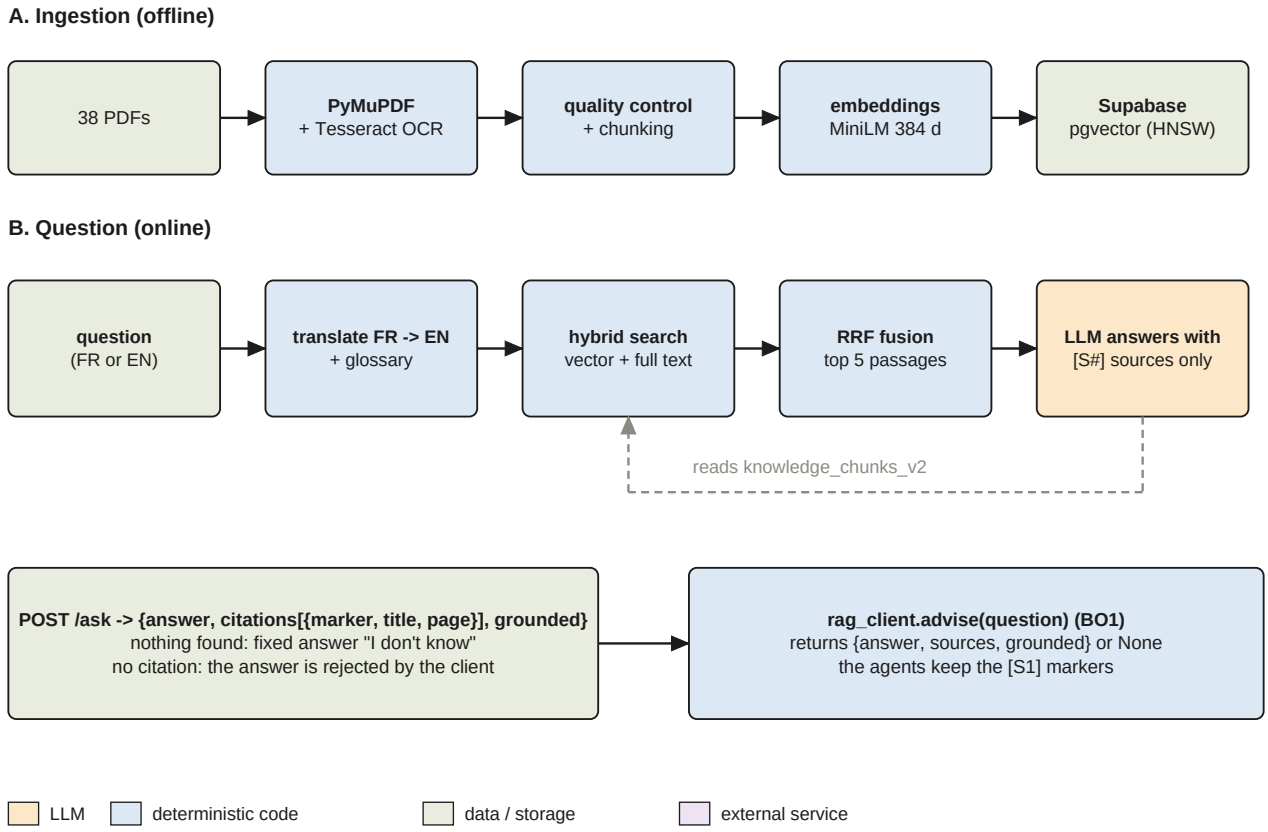


Figure 15: offline ingestion and online query.

Element	Detail
Corpus	38 PDFs: colours, fabrics, dress codes, capsules, accessories
Ingestion	PyMuPDF, Tesseract OCR for scanned books, quality control, chunking
Index	knowledge_chunks_v2, Postgres + pgvector, HNSW, cosine, 384 dimensions
Embeddings	sentence-transformers all-MiniLM-L6-v2
Search	hybrid: vectors + full text, RRF fusion, top 5 passages
Question	French translated to English with a glossary; English unchanged
Generation	answers only with numbered sources; otherwise "I don't know"
API	POST /ask, /ask/stream (SSE), API key, rate limit, history

Measured quality

Metric	Value	What it measures
Hit@5	91.67%	the right passage is in the top five
MRR	85.42%	average rank of the first correct passage
Context precision	71.67%	share of retrieved passages that are relevant
Indexed chunks	14,574	size of the pgvector index
Documents	38	processed fashion books

Integration: `rag_client.advise(question)` returns {answer, sources, grounded} or None. The AI Stylist exposes it as the **fashion_advice** tool; the Outfit Builder calls it in the **knowledge** node. [S#] markers are kept and rendered as badges (book and page).

13. Deterministic and machine-learning algorithms

The project combines several algorithm families. LLMs never replace an exact computation when one is possible.

Algorithm	Used by	Role
YOLO (dressme_best.pt)	cataloguing, live scan, similar, purchase	garment detection and segmentation
OR-Tools CP-SAT	Trip Agent	optimal suitcase selection under hard constraints
XGBoost	Gap Agent, Purchase Agent	gap ranking and purchase score
Outfit heuristics	Outfit Builder (fallback), BO4	colour 0.30 + style 0.20 + occasion 0.30 + season 0.20
FashionCLIP	Gap Agent, purchase redundancy	visual similarity between garments
CLIP ViT-B/32	Purchase Redundancy	semantic redundancy of an item
Hybrid search + RRF	RAG	fusion of vector and text rankings
Business rules	Wardrobe, Outfit, Trip, Gap	validation, constraints, guardrails

The suitcase solver

- Variables: one binary variable per wardrobe item (packed or not).
- Hard constraints: at least one outfit per day and activity, weather requirements, formality, weight limit, no duplicate role.
- Objective: minimise weight then item count while maximising possible combinations.
- If the problem is **INFEASIBLE**, `suggest_alternatives` explains the blocking constraint and never silently relaxes it.

14. Avatar and virtual try-on

The avatar is generated from a full-body photo; try-on then applies an outfit or a garment to that avatar. The provider is configured with `AVATAR_PROVIDER`.

- **3D avatar**: Hunyuan3D v3 through WaveSpeed (default) or fal.ai.
- **Full outfit try-on**: `/api/avatar/tryon-outfit` (Bria virtual try-on).
- **Shop garment try-on**: `/api/avatar/tryon-garment`.
- Generations are **background tasks**: the route returns a job_id, the interface polls the status and then shows the result.
- A missing piece detected by Find Similar can be tried on before purchase.
- No avatar image is ever sent to an LLM agent: only the image providers see it.

15. Security, guardrails and tests

Theme	Measure
Grounding	wardrobe ids only; read-only tools; numbered sources; real weather
Code validation	Outfit (composition rules), Trip (Pydantic + solver), Purchase (verdict), Similar (ids)
Graceful degradation	RAG, weather, LLM or HF unavailable: partial answer or rule-based fallback
Isolation	Supabase token verified on every route; queries filtered by user_id; A2A tasks are private
Secrets	API keys server-side only; never in the frontend or in a prompt
Honesty	the Gap Agent states that XGBoost is trained on synthetic data; the RAG says "I don't know" instead of inventing
A2A errors	failed task with a neutral message, no internal detail
Rate limiting	RAG service and sensitive routes

Tests

Suite	Count	Note
Orchestrator	101	routing, memory, simulated A2A calls
BO1 backend	85	fake LLM, fake RAG, fake weather
Frontend	30	components and flows
BO4	1,095 (25 skipped)	local graph and heuristics
Purchase engine, Trip Agent	dedicated tests	tools, solver, deterministic fallback
End to end	real trials	GPT-4o, RAG, Open-Meteo

16. Assumed limitations

- The RAG does not know the wardrobe; it only contains fashion books.
- The Gap Agent XGBoost ranker is trained on synthetic data until real feedback arrives.
- The AI Stylist has no memory between two messages; Trip Agent memory is server-side and lost on restart.
- The BO4 Stylist and the 8070 orchestrator are not wired to the React interface.
- The Purchase Agent depends on Hugging Face: without HF_TOKEN it is inactive; local Qwen2.5-3B requires enough memory and GPU.
- Weather covers one day at a time (up to 15 days) for a single city.
- A2A is a lightweight implementation: JSON-RPC and agent cards, without streaming or push notifications.
- Next steps: per-user preference memory, worn-outfit journal, fully A2A orchestrator, migrating the AI Stylist to an explicit graph.

17. Running the project and demo scenario

Step	Command / variable	Port
BO1	.\start_bo1.ps1	8000
BO3	.\start_bo3.ps1	8001
Help Me Pack	.\start_packing.ps1	8020
RAG	.\start_rag.ps1	8010
Interface	pnpm dev	8443
Standalone orchestrator	uvicorn orchestrator.main:app --port 8070	8070
Ollama	ollama serve	11434

Useful environment variables

Variable	Role
OPENAI_API_KEY	GPT-4o, gpt-4o-mini, vision
SUPABASE_URL / SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY	auth, database, storage
HF_TOKEN / QWEN_BACKEND / HF_ENDPOINT_URL	Purchase Agent (local, hf_api, auto, off)
GAP_A2A_URL	Gap Agent address for the check_gaps tool (empty disables it)
RAG_API_URL / RAG_API_KEY	agent RAG client
AVATAR_PROVIDER	wavespeed or fal
OLLAMA_HOST	local LLM for BO4 and the 8070 orchestrator

Ten-minute demo scenario

1. Add three garments by photo: show the generated record and the measured colour.
2. Open Wardrobe and ask "show me my jackets for work": the LLM chooses the filters.
3. AI Stylist: "what should I wear tomorrow evening for a dinner in Tunis?": weather, outfit, reasons, sources.
4. Outfit Builder: generate three outfits, then Shuffle for alternatives.
5. Help Me Pack: "suitcase for Paris, June 3 to 7, 8 kg": clarification question, then checklist.
6. Wardrobe Gaps: goal "winter work": ranked gaps and Tunisian shop products.
7. Should I buy this?: photograph an item and show the verdict, evidence and fallback.
8. Dashboard: statistics, outfit of the day and notifications.
9. Finish with an A2A task: `python scripts/a2a_discover.py --ask`.

18. Repository map

Path	Content
backend/app	BO1: routes, LangGraph agents, detector, cataloguing, avatar
DressMe UI Blueprint	BO3 and the React frontend
packing_backend	Trip Agent, CP-SAT solver, checklist
rag_backend	ingestion, pgvector index, /ask service
orchestrator	standalone orchestrator 8070 (graph, services.yaml)
a2a_protocol	A2A server and client, agent cards
agent_kit	shared LangGraph tool loop
bo4	local nine-node Stylist (Ollama)
smart_purchasing	Qwen + LoRA Purchase Agent and shopping tools
purchase-redundancy-ai	GPT-4.1 / Claude + CLIP redundancy service
scripts	startup, A2A discovery, maintenance
docs	technical guides and presentation PDFs

19. Glossary

Term	Definition
Agent	component combining an LLM, tools and a decision policy
LangGraph	state-graph library for agents, with nodes, edges and a checkpointer
A2A	Agent2Agent protocol: agent card + JSON-RPC tasks
RAG	retrieval-augmented generation: answer from retrieved, cited documents
Chunk	text passage indexed in pgvector
RRF	Reciprocal Rank Fusion, merging several rankings
Grounding	obligation to base every claim on real data
Guardrail	code or prompt rule that prevents an invented output
Checkpointer	mechanism that saves graph state between calls
Fallback	deterministic path when an external service is unavailable
CP-SAT	OR-Tools constraint-programming solver
LoRA	lightweight adapter that specialises a pre-trained model